UNIVERSITÀ DEGLI STUDI DI MILANO
DIPARTIMENTO DI
SCIENZE SOCIALI E POLITICHE

ALGORITHM FOR MASSIVE DATASET

Project 1: Finding similar items

Ayazhan Kaiyrgozhina

ID: 968164

DECLARATION

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Contents

1 Introduction **1**

2 Dataset Preparation **1**

2.1 Dataset description 1

2.2 Dataset preprocessing 2

2.3 SPARK 4

3 Jaccard similarity, MinHashing and LSH to find similar items **4**

3.1 Jaccard Similarity 4

3.2 MinHash 5

3.3 LSH 6

4 Results **8**

List of Figures

1 Figure working dataset 2

2 Comparison of data methods. Reprinted from Stagg, Blackshaw, Hall, et al. (2022), Figure 3. 3

3 Whitespace tokenization result 3

4 LSH implementation 7

5 Similarity score distribution 8

6 Similarity score density 9

7 Cumulative distribution 9

8 Similar pair summary 10

9 Similar pair with remapping back 10

10 Pairs by similarity bucket 11

List of Tables

1 INTRODUCTION

This report contains result of the applying finding similarity technique over scientific abstract column on the open dataset of ArXiv open public scholar articles. (Clement, Bierbaum, O'Keeffe, and Alemi, 2019).
For analyzing the 30,000 sample of data with id and abstract were taken and as a result, 799 total similar pairs were found with moderate similarity rate at mean 0.69. Jaccard similarity were applied approximated on Min-Hashing and according candidate pairs were generated by Locality Sensitive Hashing techniques.Those calculations required pairwise comparison due to complexity of data, and approximating methods were used for effective usage of memory and future scalability.

2 DATASET PREPARATION

2.1 Dataset description

The dataset is the collection of scientific unstructured articles list contains columns:

- "id" - unique id
- "abstract" - main analyzing item, based on abstract unstructured text which similarity will be measured
- "authors" - for remapping back and for data evaluations and insights
- "title" - for remapping back and for data evaluations and insights

Currently, this project aims to find similar articles by analyzing the abstract by calculating the Jaccard distance.

	id	abstract
0	0704.0001	A fully differential calculation in perturba...
1	0704.0002	We describe a new algorithm, the (k, ℓ)-...
2	0704.0003	The evolution of Earth-Moon system is descri...
3	0704.0004	We show that a determinant of Stirling cycle...
4	0704.0005	In this paper we show how to compute the $\L...

Figure 1: Figure working dataset

2.2 Dataset preprocessing

For comprehensive data preprocessing of unstructured data, I was focusing on the chart of preprocessed text data for health. However, I found this health-related article's dataset close to my project of scientific articles in terms of looking at the data structure and patterns of unstructured data characteristics, as the table shows the effectiveness of those techniques.

This chart shows the most common methods that researchers use to clean up and prepare unstructured text data for analysis. It compares how often these techniques appear in general research versus specific health science studies. The data reveals that standard cleaning steps, such as removing punctuation, filtering out common "stop words," and breaking text into individual words, are in fact most popular in those type of dataset as here, such as scientific articles, as for analyzing human writing text patterns into a structured format for further algorithms to apply.

However, in my case of application, the project, I applied preprocessing techniques on data frame, not Spark partitions, for simplicity of using such; and since I was working my project through Google Cloud VM, I had no problem with memory. However, for scalability, I find this one of the downgrades of my project implementation.

Saying all that, worth to note, since I working with only abstract I had and sample of 30,000, there were no problem of such preprocessing calculations.

For this project, namely, those preprocessing were applied:

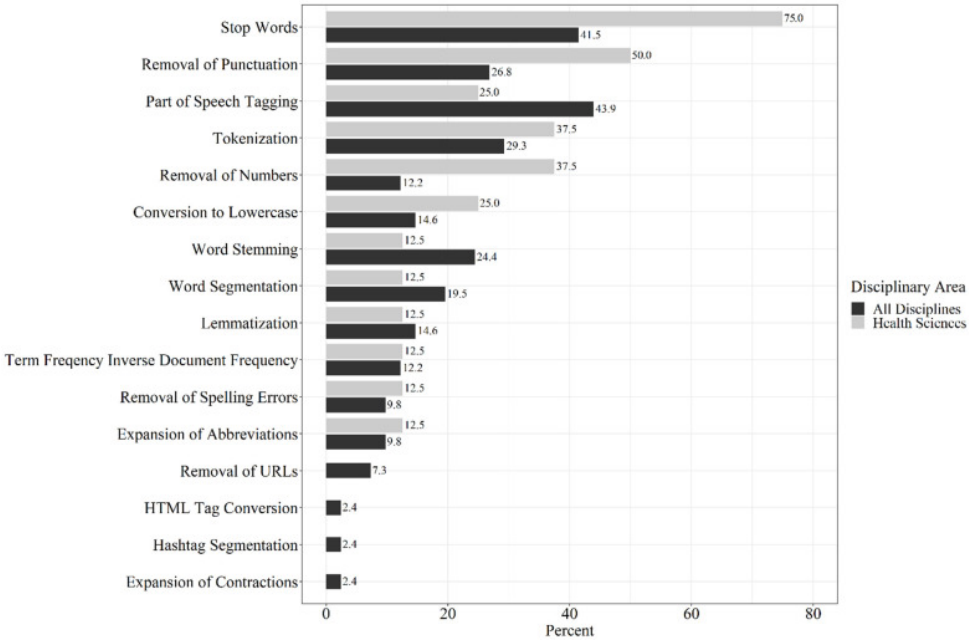

Figure 2: Comparison of data methods. Reprinted from Stagg, Blackshaw, Hall, et al. (2022), Figure 3.

- Applying lowercasing and removing URLs and email addresses. item[-] Remove scientific specific characters such as LaTeX commands, quotes, citation indices, and other entities.
- Removing UTF-8 symbols and other punctuation characters.

After simply cleaning the abstract through whitespace tokenization, which split characters based on spaces, new lines, anything empty slot; such receiving logical full "words." This especially helpful for scientific papers with less of filler words and more in common terms, and specialized glossary, lead to similarity of the context around it by the field.

```
+---------+-------------------------------------------------------------+
|       id|                                                 filtered_tokens|
+---------+-------------------------------------------------------------+
|0705.1484|[historically, the, thermodynamic, behavior, of, gasses, ...|
|0704.3661|[we, consider, controllability, of, two, conjugate, obser...|
+---------+-------------------------------------------------------------+
only showing top 2 rows
```

Figure 3: Whitespace tokenization result

2.3 SPARK

Spark is a distributed processing engine for large-scale data with optimization for in memory computations. It primarily processes data in batches using RDDs (Resilient Distributed Datasets) and DataFrames, where RDD is a low-level abstraction perfectly fit for the text material as in the project dataset. Becasue of $O(n^2)$ complexity of finding similarity algorithm, a distributed framework SPARK was required for effective calculations.

Below are some of the RDD transformations and actions used in this project:

- `map` for MinHash signatures
- `flatMap` for LSH band keys and candidate pairs
- `collectAsMap`
- `filter` for removing low-similarity pairs
- `saveAsTextFile` for sorting results

```
signatures_rdd = tokens_rdd.map(compute_sig)
print("Sample signature length:", len(signatures_rdd.first()[1]))
```

3 JACCARD SIMILARITY, MINHASHING AND LSH TO FIND SIMILAR ITEMS

3.1 Jaccard Similarity

The Jaccard similarity counts by overlapping two sets A and B. Then those coefficient is—it is the size of the intersection divided by the size of the union. So if the Jaccard similarity is 1, it is Jaccard similarity:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

- 1.0 – identical sets
- 0 – no overlap

On this project, as the sample of 30,000 rows were taken, then it would mean around 450 million pairs, which is very hard to compute; but instead, there been used:

- MinHash for approximating Jaccard via short signatures.
- LSH for grouping docs with similar signatures into bands. So then only those groups in the same band only get candidate pairs.
- Then computing Jaccard on those candidate pairs.

3.2 MinHash

For computing Jaccard similarity, first it's applied hashing functions to build short signatures for each set. MinHashing allows to convert high dimensional sparse factors into fixed by length, compressed signatures. So, the more their signatures agree, the more similar two sets they are. For each hash function (h), its taken the minimum hash value over all elements in the set. We set up the hyperparameter (K) (number of hash functions): a small (K) is faster but gives a noisier estimate, while a large (K) gives a more accurate estimate but uses more time and memory. For this project 1000 indiependent hash function has been chosed for more accuracy demand on detail specific articles.

Hash function approximates random row permutation in the characteristic matrix, so each hash function simulates one such permutation. p prime must be bigger than vocabulary size (here 53314), so the Mersenne prime were chosen. This ensures proper hashing behavior with fewer collisions. For each hash function its computed $h_i(x)$ for every element of x. Then the minimum hash values among all elements as $\min_{x \in A} h_i(x)$ to get signature vector.
Linear congruential hash function:

$$h_i(x) = (a_i \cdot x + b_i) \bmod p, \quad i = 1, \ldots, K$$

where
Then, instead of sending hash functions to every worker repeatedly, it's easier to broadcast them

once. Then to construct the Signature Matrix, which columns are MinHash signature of each document with i row and D_j document in column j under a hash h_i:

$$SM[i, j] = MinHash_i(Dj)$$

For any two documents A and B and a single hash function i, so then MinHash approximates Jaccard similarity.

$$\Pr[\text{MinHash}_i(A) = \text{MinHash}_i(B)] = J(A, B)$$

A signature is an array of values, each corresponding to the output of an hash function applied after a permutation of the rows. It's a compact representation of the characteristic matrix.

3.3 LSH

The goal of Locality Sensitive Hashing is to find pair of similar document; however, computing Jaccard similarity for all pair existed would include very dense computing and unoptimized usage of the memory. After reducing each document to a compact *k-length* signature, we then split those signatures into bands B with candidate pairs.

LSH for each document D:

generate K-bit hash code

use hash code as key to insert document into hash table

then look at the other documents that end up ont he same bucket in the haash table for collisioni

compare docs to doc sin the same bucket

(Lavrenko, 2015)

This process is enough to detect exact duplicates, which is more important for scientific document comparison; however, as one of the cons of it is missing near duplicates, because having similar hash codes but not identical goes to different bucket. So then, it's needed to repeat process with different hash tables and different hash codes. Then the Jaccard similarity computing will be based on those candidate pairs if all rows in at least one band match.

$$K = R \times B$$

where K — total length of MinHash signature

r -row his controls how many rows should in band must match to consider documents as similar; for the abstract, 5 is reasonable number to consider.

B-number of bands given the total length K

$$Pr[candidatepair] = 1(1J^R)^B$$

,

```
R = 5
B = K // R
assert B * R == K

def band_key(band_idx, band_vec):
    h = hash((band_idx, tuple(band_vec)))
    return h % (2**32)

def emit_band_keys(row):
    doc_id, sig = row
    out = []
    for i in range(B):
        start, end = i * R, (i + 1) * R
        key = band_key(i, sig[start:end])
        out.append((key, doc_id))
    return out

band_pairs = signatures_rdd.flatMap(emit_band_keys)
```

Figure 4: LSH implementation

4 RESULTS

According to the 30,000 sample were taken, the 799 similar items above set threshold has been estimated.s I said above, one of the drawbacks of the techniques applied, from mathematical point of view, is focusing on the identical in common words rather in semantics pairs were chosen; because the first as a preprocessing way, I kept text as tokens using the a single word without any punctuations as a token, rather shingles; and one of the drawbacks of LSH itself, I by setting certain K rows to consider in a band and their similarity, we set focus on identicalness rather semantic similarity.

Regarding the similarity threshold, on one hand, by following the formula $(1/B)^1/R$ the result is 0.35 however, dataset distribution of similarity, which can be shown below in similarity score distribution by median, lean more to 0.52, which is way higher than that; so the setting 0.5 as hyperparameter threshold is decided as correct approach. So, all the similar pairs that were

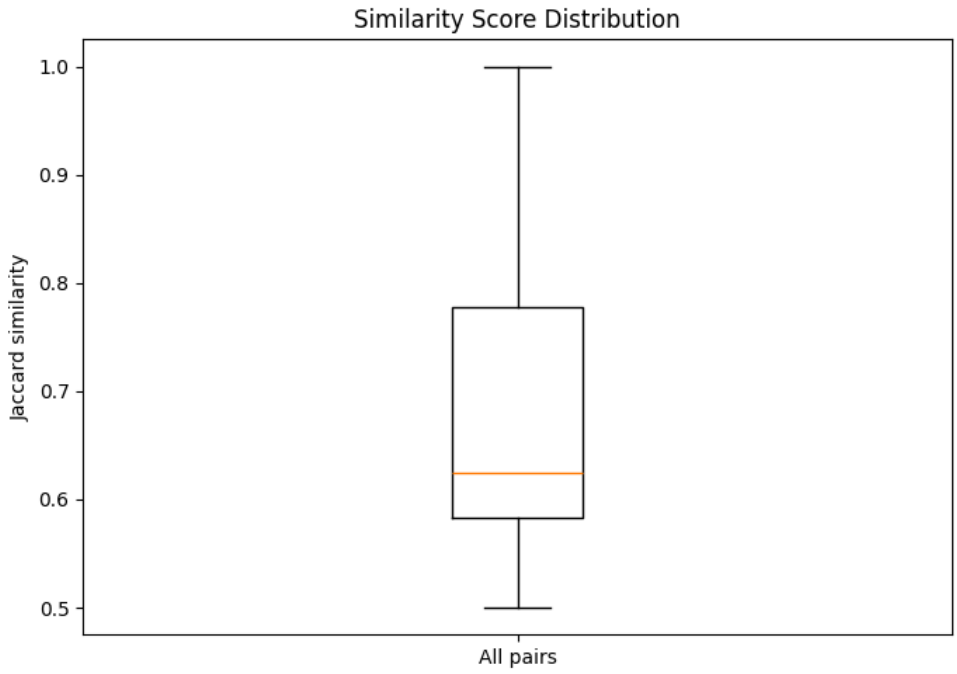

Figure 5: Similarity score distribution

above such set threshold were distributed in such way to closer to 0.5 and being less prominent on exact 1.0 Jaccard similarity.

Summarizing the results

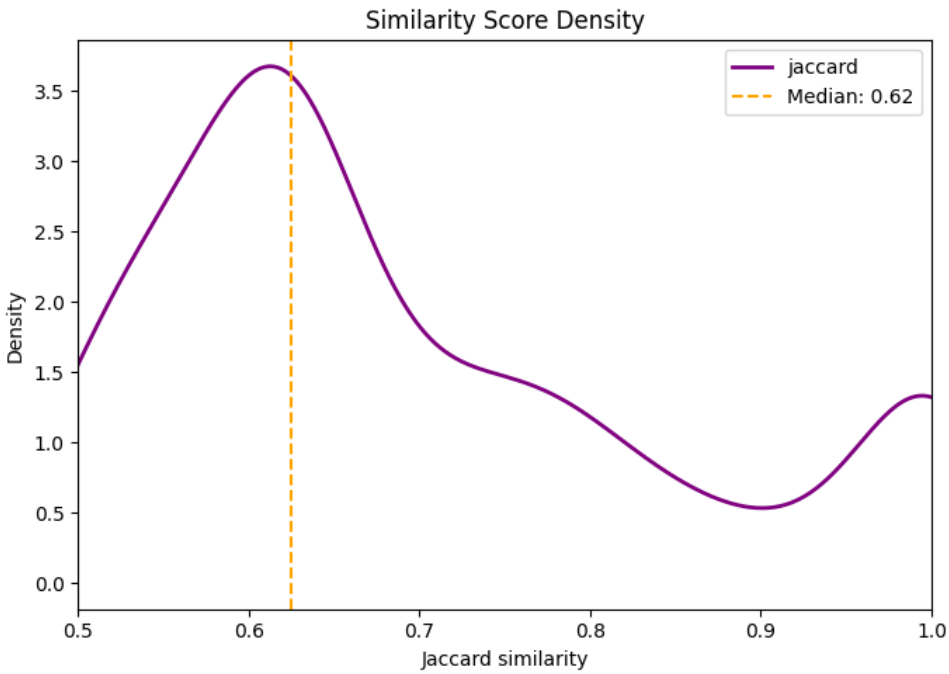

Figure 6: Similarity score density

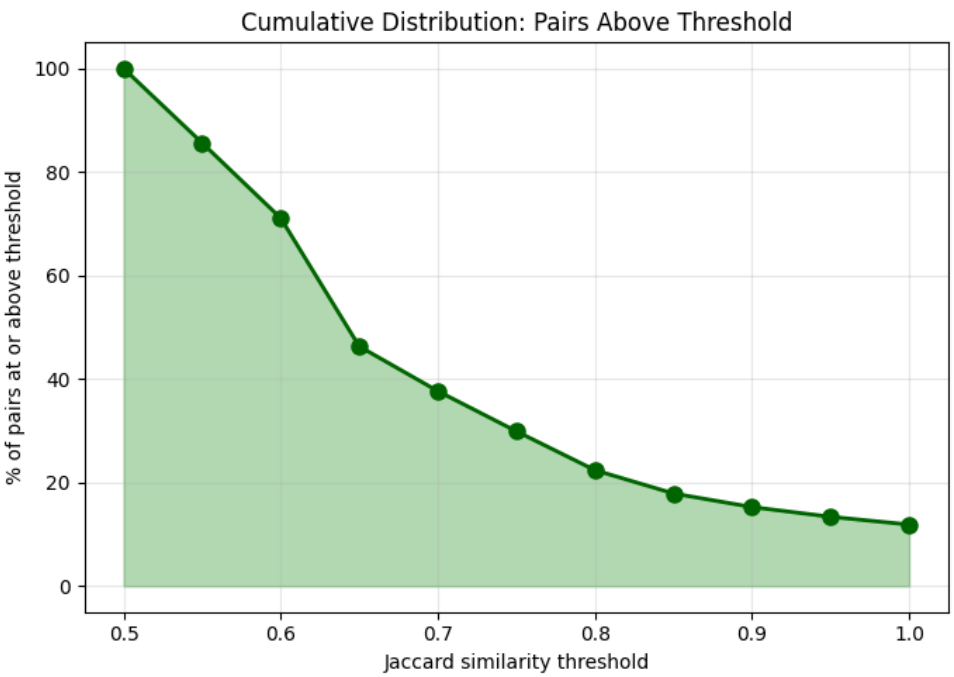

Figure 7: Cumulative distribution

	Metric	Value
0	Total similar pairs found	799
1	Unique papers in similar pairs	632
2	Mean Jaccard similarity	0.6952
3	Median Jaccard similarity	0.6250
4	Std dev	0.1533
5	Pairs with sim ≥ 0.8 (high similarity)	179
6	Pairs with sim = 1.0 (identical)	95
7	Min similarity (threshold)	0.5000

Figure 8: Similar pair summary

As an example, through remapping back to dataframe on each author and title corresponding to those id pairs of similar article abstracts will be: 1

	id1	title1	authors1	id2	title2	authors2	jaccard
0	0704.0736	p-Adic Haar multiresolution analysis	V. M. Shelkovich, M. Skopina	0705.2294	p-Adic Haar multiresolution analysis and pse...	V.M. Shelkovich, M. Skopina	0.636364
1	0710.1372	The average kinetic energy density of Cooper p...	S. Salem-Sugui Jr., M.M. Doria, A.D. Alvarenga...	0710.2459	The average kinetic energy density of Cooper p...	S. Salem-Sugui Jr., Mauro M. Doria, A. D. Alva...	0.828571
2	0704.0156	Nonlinear force-free coronal magnetic field ex...	Han He, Huaning Wang	0706.0379	Giant Magnetoresistive Effect in Colloidal Mag...	Michael P. Delmo, Kensuke Kobayashi, Shinpei Y...	1.000000
3	0706.0379	Giant Magnetoresistive Effect in Colloidal Mag...	Michael P. Delmo, Kensuke Kobayashi, Shinpei Y...	0708.4301	Zooming into the large circumstellar disk in M 17	M. Nielbock, R. Chini, V. H. Hoffmeister, D. E...	1.000000
4	0704.0156	Nonlinear force-free coronal magnetic field ex...	Han He, Huaning Wang	0707.1206	Is territory defence related to plumage orname...	V.M. Viera (DEPE-Iphc), P.M. Nolan, S.D. C\^ot...	0.777778
5	0708.0110	Metropolis simulations of the manipulation of ...	Chris J. Lee, Tim Molteno and Peter J. Manson	0708.4122	On the stability of the triangular libration p...	I. I. Shevchenko	0.500000
6	0705.1694	Note on Schmidt Decomposition in Infinite Dime...	Su Hu, and Zongwen Yu	0708.1068	Capability of multi-detector analyses on super...	Shao-Hsuan Chiu	0.615385
7	0708.1946	Vector and Axial Nucleon Form Factors:A Dualit...	A. Bodek, S. Avvakumov, R. Bradford and H. Budd	0709.3538	Extraction of the Axial Nucleon Form Factor fr...	A. Bodek, S. Avvakumov, R. Bradford, and H. Budd	0.882353
8	0705.3856	Gamma-ray albedo of the moon	Igor V. Moskalenko (Stanford), Troy A. Porter ...	0708.2742	The Gamma-ray Albedo of the Moon	Igor V. Moskalenko (Stanford), Troy A. Porter ...	0.840909
9	0705.2558	Invariance Mechanics - A new direction for qua...	Paul Bird	0708.0598	An Application of Chromatic Prototypes	Matthew McCool	0.625000

Figure 9: Similar pair with remapping back

The divide of Jaccard values into predefined similarity buckets and counting them shows this distribution of how many pairs fall into each similarity range, which we can visualize here:

In conclusion, from 30,000 samples of scientific abstract observed, 799 total similar pairs were found which mean 0.69 Jaccard similarity, which would show that this algorithm does work on both identical works such as with 1.0 similarity (and also with moderate similarity characteristics); for scientific abstract, such characteristics can be counted as unique field specific terms and meanings that algorithm could successfully capture.

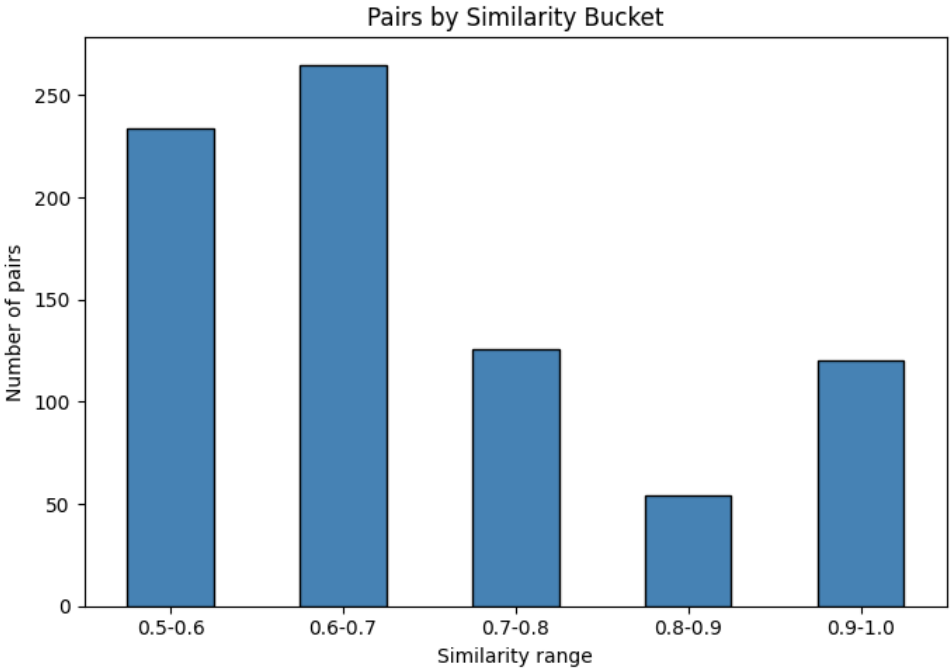

Figure 10: Pairs by similarity bucket

REFERENCES

Clement, Colin B., Matthew Bierbaum, Kevin P. O'Keeffe, and Alexander A. Alemi (2019). *On the Use of ArXiv as a Dataset*. arXiv: 1905.00075 [cs.IR].

Lavrenko, Victor (Sept. 2015). *LSH.8 Locality-sensitive hashing: the idea*. YouTube. Accessed: 2024-05-20.

Stagg, Helen R., Leanne Blackshaw, Lawrence Hall, et al. (2022). "A systematic review of data restructuring and reorganising methods for health research". In: *International Journal of Population Data Science* 7.1, p. 1757. DOI: 10.23889/ijpds.v6i1.1757.